

Task 1

For this task, I used the sentence-transformers library. This library is built using pytorch and Hugging Face's Transformers which provides a high-level API to quickly load the transformer base models that are optimized for sentence embeddings.

Architecture System

- I used mean pooling in the pooling strategy over the token embeddings from the last hidden state. This method would find the average all token embeddings, weighted by the attention mask.
- I also used fixed-length embeddings which transform all sentences into a vector length of 384 regardless of the original token length. This is a very important step in clustering algorithms.
- The chosen library automatically applied L2 normalization which is useful for cosine similarities. This is beneficial in the downstream.
- Tokenization is handled internally by the Hugging Face tokenizer associated with MiniLM. It automatically pads and truncates inputs to the model's maximum sequence length.

Task 2

In this task, I expanded the architecture to support a multi-task learning setting by introducing two supervised classification tasks A and B. Task A is a sentence topic classification and for Task B I used sentiment Analysis. I used Sentiment Analysis because it is a sentence-level classification but with different label sets such as positive, neutral, or negative. As for the purpose of this project, it would be easier to use to create fake lables and intuitive to interpret.

In this task each head is implemented as a fully connected linear layer that maps the pooled sentence embedding to 3 3-class output space. Each head produces its own logic during the training, which are compared with the ground truth labels using a separate cross-entropy loss.

For loss calculation, I used the sum of the task-specific losses which allows the model to learn both task at the same time instead of working on them individually. This approach shares the same sentence encoder that enables the model to generalize shared features across tasks while still maintaining task-specific decisions. This architecture efficiently supports multi-task learning without requiring multiple separate models.

Comparing with task 1 which the resulting embeddings were not directly used for any downstream supervised task; rather, they served as general-purpose sentence representations for potential use in clustering, similarity, or other unsupervised applications. No classification

heads or loss functions were applied in Task 1, and the model was not trained or fine-tuned. It was purely used in inference mode.

Task 3

Part A

If the entire network should be frozen:

Pros: Freezing the entire network turns the model into a static feature extractor. This means no weights are updated during the training. This can be very useful when you are only embedding for downstream use and not classification. When you have very limited labeled data and you want to avoid overfitting, freezing the network can help a lot. The last scenario that freezing the network can be useful when we want fast inference without retraining the whole model again.

Cons: In this model, there is no training happening. The model won't adapt to the new tasks or datasets' performance, which can act poorly on classification tasks like Task 2.

If only the transformer backbone should be frozen.

Pros: This method is very common strategy in transfer learning. This method retains the powerful sentence representation learned by the transformer and only train the task-specific heads.

This method can help with

- a. reducing the training time and memory usage.
- b. Prevent overfitting on a small dataset.
- c. Ensure the backbone retains general linguistic knowledge

Cons: With this method, the model won't be able to adapt the core representation to domain-specific data. If the new dataset is significantly different from what the transformer was trained the whole model's performance will suffer.

If only one of the task-specific heads (either for Task A or Task B) should be frozen.

Pros: With this method of the tasks have already been well-trained, and you want to preserve learned knowledge in one task while updating the other one.

Cons: This method can cause task interference during shared encoding updates. The frozen head won't adapt if the encoder changes due to the training from the other task.

Part B

The choice of a pre-trained model.

I would select a model with a general purpose such as all-Mini_L6-v2 since the concept of the model is not defined completely. This model would help with most of the tasks and performs well.

For domain-specific tasks, I would go with more specialized models like BERT substituted, such as SciBERT or LegalBERT.

The layers you would freeze/unfreeze.

Freezing

- a. Freeze the transformers to the backbone or the pre-trained encoder
- b. This would help to train only the task-specific heads such as sentiment analysis

Unfreezing

- a. Unfreeze the transformer layers after the initial training
 - b. Fine tune the entire model so it can adapt with deeper tasks
3. The rationale behind these choices.
- This approach prevents the forgetting issue of the pre-trained knowledge in the early stages of the training.
 - It would also enable the classification heads to first learn task-specific patterns independently.
 - Gradual unfreezing would allow the model to refine the internal representations for the new tasks
 - Using discriminative learning rates would ensure a controlled fine-tuning. Discriminative learning can be LR for the backbone or higher LR for the heads.

Task 3 and 4 documentation

In expanding the model to support multi-task learning, I used two supervised tasks—sentence topic classification and sentiment analysis built on top of a shared transformer encoder. For

training strategy (Task 3), I used different freezing scenarios and adopted a phased fine-tuning approach: I initially froze the transformer backbone to allow the task-specific heads to learn independently, then gradually unfroze the backbone to fine-tune the entire model. This helped to retain the general language knowledge from the pre-trained model while still allowing adaptation to new tasks.

I used a pre-trained model, all-MiniLM-L6-v2, to leverage strong sentence embeddings and accelerate learning. The model was trained using separate cross-entropy losses for each task, which were combined and backpropagated through the shared encoder. I tracked both accuracy and F1 score to understand the model and task performance better which is important in low-resource or imbalanced scenarios. Over multiple epochs, the model showed clear improvement in both tasks, demonstrating the effectiveness of shared representation learning in a multi-task setting. These results validate the design choices and show how transfer learning can be strategically applied to maximize both efficiency and performance.